

PS015 – Enterprise Talent Acquisition & Tracking Portal

Comprehensive Backend Microservices Implementation & Postman Verification Guide

Branch: auth-profile | Java 21 | Spring Boot 3.3.4/3.3.6 | Spring Cloud 2023 | PostgreSQL 18

1. Project Overview

PS015 is an enterprise-grade, cloud-native talent acquisition and recruitment management portal engineered with Spring Boot microservices. It is architected for scalability, role isolation, resilient inter-service communication, and high-security compliance. The platform automates end-to-end recruitment pipelines from candidate registration, resume profiling, and job publication to application status management, candidate shortlisting, rejection, and final selection.

2. Problem Statement

Modern enterprise hiring requires secure separation of roles (Candidates vs. HR Recruiters vs. System Administrators), independent service scaling, unified routing, dynamic service discovery, resilient cross-service validation (preventing invalid or orphan applications), and tamper-proof authentication across distributed APIs.

3. Technology Stack

Layer / Component	Selected Technology & Specification
Programming Language	Java 21 (OpenJDK 21.0.9 LTS 64-Bit)
Framework	Spring Boot 3.3.4 (Auth, Profile) / 3.3.5 (Application) / 3.3.6 (Job, Gateway, Eureka)
Cloud & Routing	Spring Cloud Gateway (Reactive), Netflix Eureka Server (2023.0.3/2023.0.4)
Security & Auth	Spring Security 6, JWT (io.jsonwebtoken 0.12.6 / 0.11.5), BCrypt Hashing
Persistence & ORM	Spring Data JPA, Hibernate ORM, PostgreSQL Driver
Database Engine	PostgreSQL 18 (Localhost :5432, ps015_auth, ps015_profile, jobdb, application_db)
Inter-Service Comms	Spring Cloud OpenFeign (Declarative REST Clients with Eureka discovery)
Build Automation	Apache Maven 3.9.6, Lombok, Jakarta Bean Validation 3.0

4. Complete Microservices Architecture

```
+-----+ | CLIENTS & POSTMAN |
+-----+ | HTTP Requests (:8080)
v +-----+ | API GATEWAY (Spring
Cloud Gateway :8080) | | /api/auth/** --> lb://AUTH-SERVICE | | /api/profiles/** --> lb://PROFILE-SERVICE | |
/api/jobs/** --> lb://JOB-SERVICE | | /api/applications --> lb://APPLICATION-SERVICE |
+-----+ | | | | | | v v v v
+-----+ +-----+ +-----+ +-----+ | AUTH-SERVICE | | PROFILE-SERVICE | |
JOB-SERVICE | | APPLICATION- | | (:8081) | | (:8082) | | (:8084) | | SERVICE (:8083)| +-----+
+-----+ +-----+ +-----+ | | | | | | (Feign Job Info)| | (Feign Profile) |
+-----+ | | | v v v v PostgreSQL: PostgreSQL: PostgreSQL: PostgreSQL:
ps015_auth ps015_profile jobdb application_db [ SERVICE DISCOVERY REGISTRY: EUREKA SERVER running at
http://localhost:8761/eureka ]
```

5. Service Responsibility Table & 6. Port Configuration

Service	Port	Database	Core Responsibilities
eureka-server	8761	None	Central dynamic service discovery registry and health heartbeat tracking.
api-gateway	8080	None	Unified entry point, non-blocking path routing, Eureka-based load balancing, global CORS.
auth-service	8081	ps015_auth	Registration, BCrypt password hashing, JWT token issuance, role-based auth endpoints.
profile-service	8082	ps015_profile	Candidate and HR profile lifecycle (CRUD), skills, experience, resume tracking, ownership enforcement.
application-service	8083	application_db	Job applications, status workflow (APPLIED, SHORTLISTED, REJECTED, SELECTED), OpenFeign validation.
job-service	8084	jobdb	Job listing management, searching, criteria filtering, HR job post management.

7. PostgreSQL Database Configuration

Each microservice maintains strict schema isolation using dedicated PostgreSQL databases on host localhost:5432. Passwords and sensitive secrets are loaded via environment variables with safe development defaults:

- **auth-service:** jdbc:postgresql://localhost:5432/ps015_auth (User: \${DB_USERNAME:postgres}, Password: \${DB_PASSWORD})
- **profile-service:** jdbc:postgresql://localhost:5432/ps015_profile (User: \${DB_USERNAME:postgres}, Password: \${DB_PASSWORD})
- **job-service:** jdbc:postgresql://localhost:5432/jobdb (User: \${DB_USERNAME:postgres}, Password: \${DB_PASSWORD})
- **application-service:** jdbc:postgresql://localhost:5432/application_db (User: \${DB_USERNAME:postgres}, Password: \${DB_PASSWORD})

8. Eureka Service Discovery & 16. Eureka Configuration

The Eureka Server coordinates dynamic discovery at <http://localhost:8761/eureka/>. Client microservices declare `eureka.client.service-url.defaultZone` and `auto-register` on startup. Self-preservation is disabled in development mode to ensure that stopped instances are immediately purged.

9. API Gateway Architecture & 17. API Gateway Routes

Spring Cloud Gateway serves as the single reverse proxy at port **8080**. Routes are configured in `application.yml` with client-side load balancing (`lb://`):

- Path=`/api/auth/**` → `lb://AUTH-SERVICE`
- Path=`/api/profiles/**,/api/profile/**` → `lb://PROFILE-SERVICE`
- Path=`/api/jobs/**` → `lb://JOB-SERVICE`
- Path=`/api/applications/**` → `lb://APPLICATION-SERVICE`

10. JWT Authentication Flow & 11. Role-Based Authorization

Authentication is stateless. The secret key is shared across microservices (404E635266556A586E3272357538782F413F4428472B4B6250645367566B5970). Claims include: `sub` (email), `userId` (numeric), `role` (CANDIDATE, HR, ADMIN), and `exp` (24 hours).

Authorization is strictly enforced using Spring Security filters. Endpoints verify roles (e.g. `hasRole('CANDIDATE')`, `hasRole('HR')`) or resource ownership (a candidate can only view/modify their own profile and applications; an HR user can view all candidate profiles and manage applications for their jobs).

18. Inter-Service Communication (OpenFeign Architecture)

When a candidate submits a job application (POST /api/applications), application-service executes synchronous inter-service checks via OpenFeign before persisting:

1. **Job Verification:** JobServiceClient.getJobById(jobId) calls job-service to confirm the job listing exists and has status OPEN. If closed or missing, an appropriate exception is returned.
2. **Candidate Profile Verification:** ProfileServiceClient.getProfileByCandidateId(candidateId) calls profile-service at /api/profiles/candidate/{candidateId} to confirm the candidate possesses a valid profile.
3. **Duplicate Prevention:** The repository ensures a candidate cannot apply to the same job twice.

19. Complete Candidate Workflow & 20. Complete HR Workflow

Candidate Workflow: Register (POST /api/auth/register) → Login (POST /api/auth/login) → Create Profile (POST /api/profiles) → View Jobs (GET /api/jobs) → Apply for Job (POST /api/applications) → Track My Applications (GET /api/applications/my).

HR Workflow: Register HR (POST /api/auth/register) → Login (POST /api/auth/login) → Create HR Profile (POST /api/profiles) → Post Job Listing (POST /api/jobs) → Review Applications (GET /api/applications/job/{id}) → Shortlist/Reject/Select (POST /api/applications/{id}/shortlist).

12-15 & 23-26. Complete Microservices API Reference & Request Examples

Method	Endpoint	Auth	Role	Request Body / Params	Status
POST	/api/auth/register	No	Any	RegistrationRequest JSON	201
POST	/api/auth/login	No	Any	LoginRequest JSON	200
GET	/api/auth/candidate	Yes	CANDIDATE	None	200
GET	/api/auth/hr	Yes	HR	None	200
GET	/api/auth/admin	Yes	ADMIN	None	200
POST	/api/profiles	Yes	CANDIDATE/HR	CreateProfileRequest JSON	201
GET	/api/profiles/me	Yes	Any (Owner)	None	200
GET	/api/profiles/{id}	Yes	Owner/HR/ADMIN	None	200
GET	/api/profiles/candidate/{id}	Yes	Feign/Internal	None	200
PUT	/api/profiles/{id}	Yes	Owner/ADMIN	UpdateProfileRequest JSON	200
DELETE	/api/profiles/{id}	Yes	Owner/ADMIN	None	200
POST	/api/jobs	No/HR	HR/ADMIN	JobRequestDto JSON	201
GET	/api/jobs	No	Any	None	200
GET	/api/jobs/{id}	No	Any	None	200
PUT	/api/jobs/{id}	No/HR	HR/ADMIN	JobRequestDto JSON	200
PATCH	/api/jobs/{id}/status	No/HR	HR/ADMIN	JobStatusUpdateDto JSON	200
GET	/api/jobs/search	No	Any	?keyword=string	200
GET	/api/jobs/filter	No	Any	?location=&type;=&minSalary;=	200
GET	/api/jobs/hr/{hrId}	No/HR	HR/ADMIN	None	200
DELETE	/api/jobs/{id}	No/HR	HR/ADMIN	None	200
POST	/api/applications	Yes	CANDIDATE	ApplyJobRequest JSON	201
GET	/api/applications/my	Yes	CANDIDATE	None	200
GET	/api/applications/{id}	Yes	Owner/HR/ADMIN	None	200
GET	/api/applications/job/{jobId}	Yes	HR Owner/ADMIN	None	200
PUT	/api/applications/{id}/status	Yes	HR/ADMIN	ApplicationStatusUpdateRequest	200
POST	/api/applications/{id}/shortlist	Yes	HR/ADMIN	None	200
POST	/api/applications/{id}/reject	Yes	HR/ADMIN	None	200
POST	/api/applications/{id}/select	Yes	HR/ADMIN	None	200

Exact Request JSON Payloads

RegistrationRequest (POST /api/auth/register):

```
{ "name": "Alice Candidate", "email": "alice@example.com", "password": "Password123", "role": "CANDIDATE" }
```

CreateProfileRequest - Candidate (POST /api/profiles):

```
{ "name": "Alice Candidate", "phone": "+91-9876543210", "profileType": "CANDIDATE", "location": "Bengaluru, India", "skills": ["Java", "Spring Boot", "PostgreSQL", "Docker"], "education": "B.Tech Computer Science", "experience": "3 years Software Engineer", "resumeHeadline": "Senior Java Developer", "resumeUrl": "https://storage.portal.com/resumes/alice.pdf" }
```

JobRequestDto (POST /api/jobs):

```
{ "title": "Senior Java Microservices Architect", "description": "Design and build enterprise cloud recruitment systems.", "company": "TechCorp Global", "location": "Bengaluru, India", "employmentType": "FULL_TIME", "experienceRequired": "5+ years", "salary": 2400000.0, "skills": "Java 21, Spring Boot, Spring Cloud, PostgreSQL", "closingDate": "2026-12-31", "hrId": 2, "status": "OPEN" }
```

ApplyJobRequest (POST /api/applications):

```
{ "jobId": 1, "coverLetter": "Experienced in Spring Boot microservices and cloud deployments." }
```

21. Postman Environment Variables & 22. Testing Instructions

Import the Postman collection and configure environment variables with `gatewayUrl=http://localhost:8080`. Use the visual Postman runner or manual sequence to execute candidate registration, JWT token capture (using Tests script: `pm.environment.set('jwtToken', pm.response.json().token);`), and automated authorization passing.

27-31. Headers, Status Codes, Validation & Error Handling

- **Authorization Header:** Authorization: Bearer <JWT>
- **HTTP 200 OK:** Successful fetch, update, search, status change, or deletion.
- **HTTP 201 Created:** User registration, profile creation, job creation, application submission.
- **HTTP 400 Bad Request:** Missing or blank required fields, password < 6 chars, negative salary, closed job applications.
- **HTTP 401 Unauthorized:** Missing token, malformed signature, expired token, or invalid email/password.
- **HTTP 403 Forbidden:** Insufficient role permissions or modifying another user's profile/applications.
- **HTTP 404 Not Found:** User, profile, job, or application ID does not exist.
- **HTTP 409 Conflict:** Duplicate email registration, duplicate profile for user, or duplicate job application.

32. Test Results & 33. Build Results

Service	Build Result	Tests Passed	Tests Failed	Key Resolved Issues
eureka-server	BUILD SUCCESS	1	0	Discovery server startup and registry initialization verified.
api-gateway	BUILD SUCCESS	1	0	Added /api/profiles/** path route for profile-service load balancing.
auth-service	BUILD SUCCESS	15	0	Added Eureka client dependency, application.name, and Eureka discovery config.
profile-service	BUILD SUCCESS	13	0	Added Eureka client, candidateId/fullName aliases, and /candidate/{id} Feign endpoint.
job-service	BUILD SUCCESS	21	0	Resolved port collision by reassigning to 8084; aligned DB password default to root.
application-service	BUILD SUCCESS	80	0	Added @JsonProperty('data') unwrap in JobResponseDto for Feign ApiResponse unwrap.

34. Known Issues & Operational Recommendations

1. **Startup Order:** Always start eureka-server (8761) first, followed by PostgreSQL databases, then the backend services (8081, 8082, 8083, 8084), and finally api-gateway (8080). Allow 10-15 seconds for Eureka discovery propagation.
2. **Database Passwords:** Local database environments should define the DB_PASSWORD environment variable if PostgreSQL uses credentials other than root.

35. Final Verified Architecture Diagram

```

+-----+ | POSTMAN TEST
COLLECTION & WEB/MOBILE CLIENT INTERFACES |
+-----+ | [PORT 8080] v
+-----+ | SPRING CLOUD API
GATEWAY (:8080) | | * CORS Policy Enabled * Reverse Proxy Routing * Dynamic Eureka Routing |
+-----+ | | | v v v v
+-----+ +-----+ +-----+ +-----+ +-----+ | AUTH-SERVICE | |PROFILE-SERVICE| |
JOB-SERVICE | | APPLICATION- | | (:8081) | | (:8082) | | (:8084) | | SERVICE(:8083)| +-----+
+-----+ +-----+ +-----+ +-----+ | * Register | | * Profile CRUD| | * Post Jobs | | * Apply Job
| | * Login/JWT | | * Candidate/HR| | * Search/Filter | * Feign Check | | * RBAC Roles | | * Feign Lookup| | * Feign
Lookup| | * Status Track| +-----+ +-----+ +-----+ +-----+ | | | v v v v
[PostgreSQL: [PostgreSQL: [PostgreSQL: [PostgreSQL: ps015_auth] ps015_profile] jobdb] application_db]
+-----+ | EUREKA DISCOVERY
SERVER (:8761) - Heartbeats & Instance Registry |
+-----+

```